

Paged- Attention

Alis

<https://softmaxxer101.github.io/>
https://x.com/import_alis

Date: 23rd September 2026

Abstract- Large language model inference relies on the KV cache during token generation, but this cache can consume significant GPU memory and suffer from fragmentation. This article explains how vLLM uses paged KV caching to divide the cache into reusable fixed-size blocks and map logical blocks to physical GPU locations through a block table. It also explains paged attention, which processes these scattered blocks in logical order using a running softmax calculation. Together, these techniques improve memory utilization, enable KV-cache sharing, and support more concurrent requests.

Index Terms- Autoregressive, inference, KV cache, LLM inference, Paged attention, Paged KV caching

I. FROM LLM INFERENCE TO PAGED KV CACHING

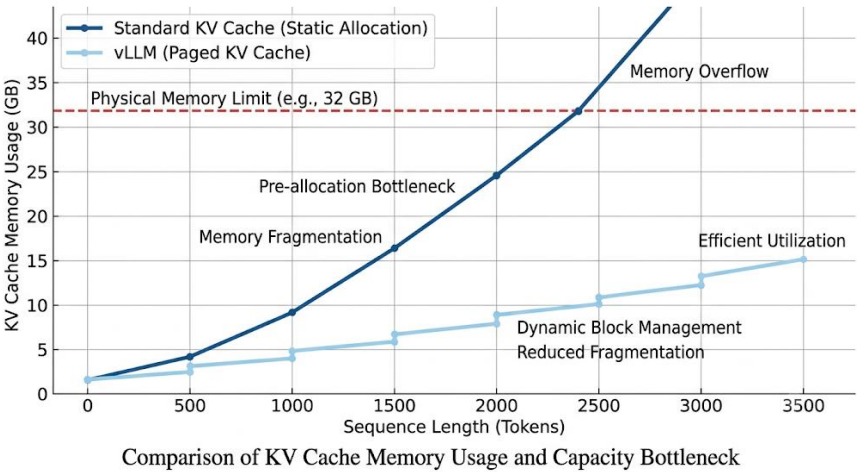
When a trained LLM receives a prompt, inference is done on the model's learned parameters to generate a sequence of output tokens. Unlike training, where the model process training data set and updates its weight, inference makes these model's learning frozen and uses these learned data for prediction.

For an autoregressive generation, inference is divided into two phases- prefill and decode. Prefill is the process where the model reads the prompt and understands it. During Decode , each newly generated tokens visits its previously computed keys and values.

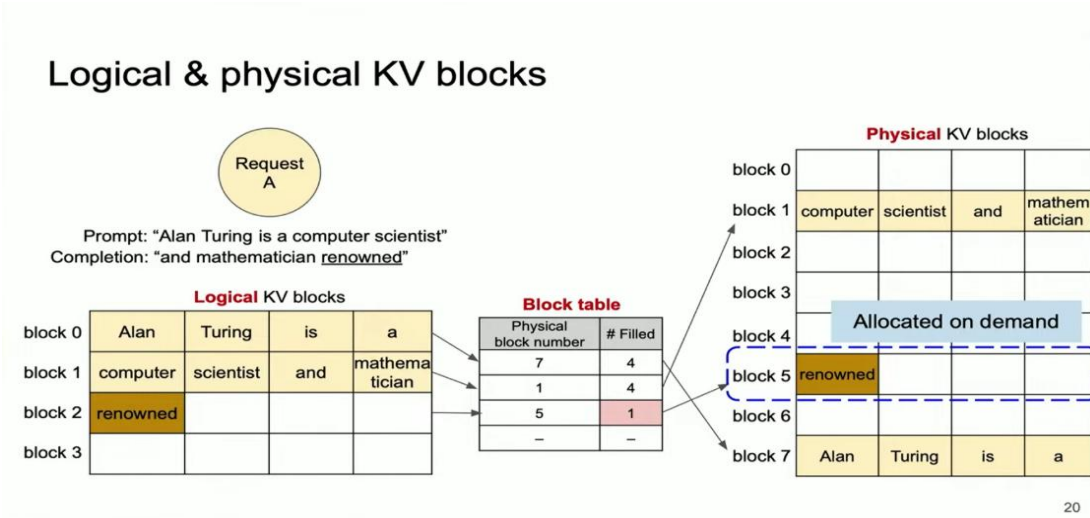
Recomputing these keys and values at every step was tiresome, and hence the idea of KV cache was introduced. Read more about KV Cache here

II. THE KV CACHE BOTTLENECK

The major bottleneck faced during LLM inference is the KV Cache memory. vLLM introduced a new technique named Paged KV Caching and Paged Attention to solve this KV Cache memory issue. The conventional contiguous KV Cache wastes GPU memory due to its internal and external fragmentation. The Paged attention solves this fragmentation and inefficient memory utilization problems.



III. THE CORE IDEA BEHIND PAGED ATTENTION

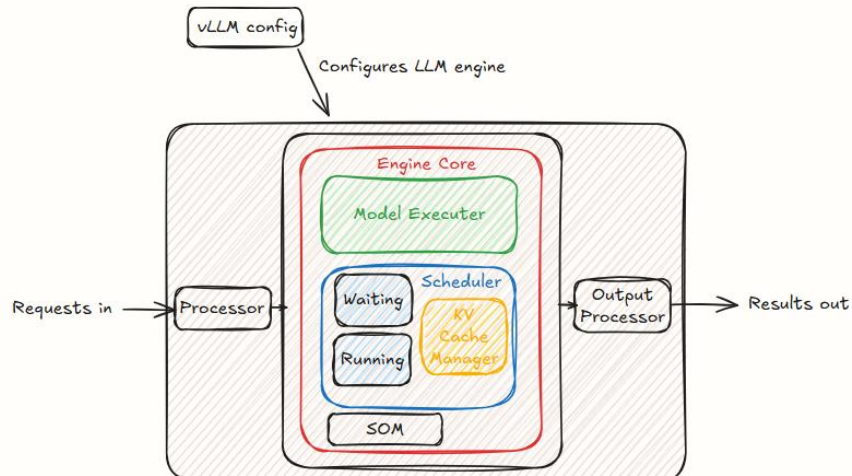


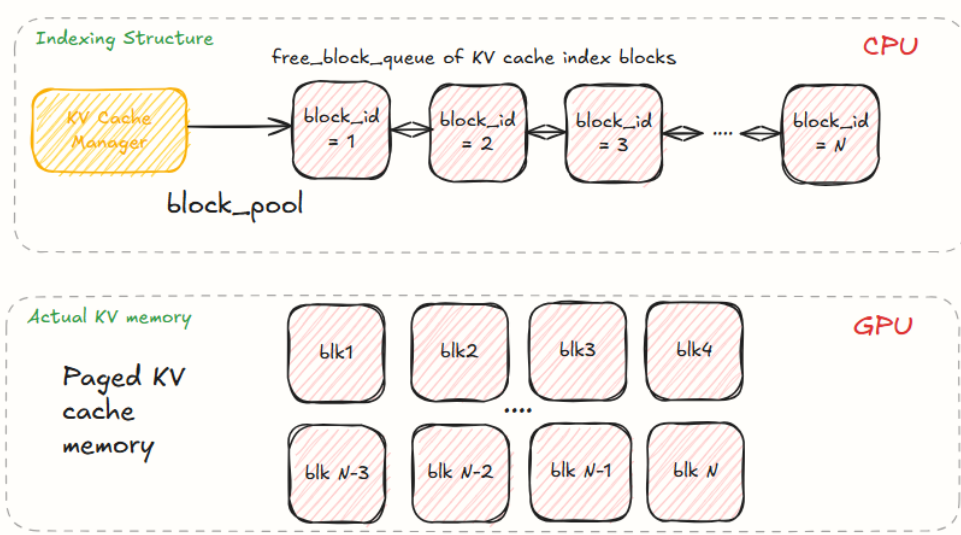
Paged Attention addresses this problem by applying the idea of virtual memory paging same as what the computer's memory management system uses. Instead of storing all the KV Cache generated in one contiguous tensor, we introduce the Logical KV blocks and the physical KV blocks. The logical KV block consist of blocks, each block can store tokens (tokens per block). Here the request are stored as it is in a contiguous manner. Then we have the Physical KV block which splits the KV Cache into non-contiguous fixed-size blocks instead of one large tensor. The concept of block table is introduced which maps the logical KV blocks to physical KV blocks. The block table consists of physical block number which maps the logical kv blocks to physical kv blocks through fields-physical block number and #filled (refer to the diagram above). This mapping behaves like many-to-one mapping between logical--->physical blocks and enables KV sharing between parallel similar prompts. This efficient packing of sequences into memory allows more concurrent requests than the old traditional.

IV. ATOMIC VIEW INSIDE "HOW PAGED KV CACHING ACTUALLY WORKS INSIDE VLLM?"

Inside this vLLM configuration, Our main focus would be the blue highlighted box- The "Scheduler box". It behaves like a commander and commands the KV Cache manager.

The KV Cache Manager allocates blocks, maintains mappings through block table and allocates and deallocates blocks. In the previous fig. we saw how the logical kv blocks are mapped to physical kv blocks using block table. But let me break it for you, there's more to it!





The GPU VRAM consists of physical kv blocks and the global free block pool. The global free block pool refers to all the unallocated free blocks which are called upon request demand. The global free block is situated inside the CPU, it is an indexing structure that stores the list of available block identifiers which is stored in the form of doubly linked list. These available block identifiers points to the free blocks present in the Actual KV memory in the GPU. The allocated and unallocated (free blocks) are present in the GPU memory together. Whenever a request comes, the KV Cache Manager fetches one identifier from the free pool and writes them into the request's block table. The free block is then allocated to the request on demand, after the completion of demand, the block is freed and returned back to the global free block pool.

During decode, new tokens are written into the next free slots, hence one block can be fetched by more than one request. The reference count refers to number of requests per block.

V. PAGED ATTENTION

Paged Attention is the mathematical side of paged KV Cache that makes it come into life and work. The traditional attention formula computes keys and values for a request in one contiguous buffer. Paged Attention solves this problem totally.

$$K_j = [k_{(j-1)B+1}, ..., k_{jB}], \quad V_j = [v_{(j-1)B+1}, ..., v_{jB}].$$

The blocks are numbered as $j=1,2,3,4,.....$.Block j refers to the collection of list of keys and values in an array.

Here B = tokens per block

K = all key vectors

V = all value vectors

j = block number

$(j-1)B+1$ = the first token in block j

jB = the last token in block j

Example with $B = 4$: (Here $j=1,2,3$)

Block 1: tokens 1–4

Block 2: tokens 5–8

Block 3: tokens 9–12

Normal Attention Formula:

$$q_i^\top K_j / \sqrt{d},$$

Running Softmax function:

$$\text{weight} = \frac{e^{\text{score}}}{\sum e^{\text{all scores}}}$$

Attention-weight vector for one block:

$$A_{ij} = (a_{i,(j-1)B+1}, \dots, a_{i,jB})$$

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t / \sqrt{d})},$$

A_{ij} contains the attention weights for tokens inside block j , when calculating attention for token i .

Final Attention output:

$$o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top.$$

This means:

1. Take the values in each block.
2. Multiply each value vector by its attention weight.
3. Add the results from every block.
4. Produce o_i , the final attention output for the current token.

In ordinary words:

output =
weighted values from block 1
+ weighted values from block 2
+ weighted values from block 3
+ ...

The block table may be initially scattered in the GPU memory but the block table lets the kernel find them and combines them in the correct logical order and performs the Paged-Attention over it. Summarizing it - paged attention is the entire process of reading those scattered blocks and computing attention over them.

REFERENCES

- [1] CloudDude, “vLLM for beginners: Key Features & Performance Optimization (Part II),” *CloudThrill*, Jul. 2, 2025. [Online]. Available: <https://cloudthrill.ca/what-is-vllm-features>
- [2] vLLM Project, “block_pool.py,” *vLLM GitHub Repository*. [Online]. Available: https://github.com/vllm-project/vllm/blob/main/vllm/v1/core/block_pool.py
- [3] vLLM Project, “Paged Attention,” *vLLM Documentation*. [Online]. Available: https://docs.vllm.ai/en/latest/design/paged_attention/